DAY-21

PYTHON 90 DAYS CHALLENGE

Study Notes | Pages 219?229

Chapter 12. Scope and Namespaces in Python

Scope means the area of a program where a variable can be accessed.

Namespace means a place where names are stored and mapped to objects.

name -> object/value

Example: x = 10

Here, Python stores the name x and connects it to the value 10.

12.1 Local Scope

A **local scope** is created inside a function. Variables created inside a function are local variables.

Syntax

```python
def function_name():
    variable_name = value
```

Explanation

1. A local variable is created inside a function.
2. It can be used only inside that function.
3. It cannot be accessed directly outside the function.
4. Local variables are created when the function is called.
5. They are destroyed after the function finishes.

Example 1

```
PYTHON CODE
def show_name():
    name = "Aman"
    print(name)
show_name()
```

Output: Aman

Invalid outside access:

```
PYTHON CODE
print(name)
```

Output: NameError: name 'name' is not defined.

deWithNishchal

12.2 Global Scope

A **global scope** is the main area of the program. Variables created outside all functions are global variables.

Syntax

```
PYTHON CODE
variable_name = value


def function_name():
    statement
```

Explanation

1. A global variable is created outside functions.
2. It can be accessed inside functions.
3. It can also be accessed outside functions.
4. Reading a global variable inside a function does not require the global keyword.
5. To modify a global variable inside a function, we need the global keyword.

Example 1

```
PYTHON CODE
course = "Python"


def show_course():
    print(course)


show_course()
print(course)
```

Output:
Python
Python

NOTE: Global variables are accessible throughout the file after they are defined.

12.3 Local vs Global Scope

Point	Local Scope	Global Scope
Created where?	Inside function	Outside functions
Accessible where?	Only inside that function	Inside and outside functions
Lifetime	Exists during function call	Exists while program runs
Example	name = "Aman" inside function	course = "Python" outside function

12.4 Variable Shadowing

Variable shadowing happens when a local variable has the same name as a global variable.

Explanation

1. If a local and global variable have the same name, Python uses the local variable inside the function.
2. The global variable is not changed.
3. This is called shadowing.

Example 1

```
PYTHON CODE
name = "Global Aman"

def show_name():
    name = "Local Aman"
    print(name)

show_name()
print(name)
```

Output:

Local Aman
Global Aman

NOTE: Inside the function, the local variable gets priority over the global variable.

WithNishchal

12.5 global Keyword

The global keyword is used to modify a global variable inside a function.

Syntax

global variable_name

Explanation

1. Use global when you want to assign a new value to a global variable inside a function.
2. Without global, assignment inside a function creates a local variable.
3. Reading a global variable does not need global. Using too many global variables is not recommended.

Example 1

PYTHON CODE

```
count = 0

def increase_count():
    global count
    count += 1

increase_count()
print(count)
```

output: 1

Without global

PYTHON CODE

```
count = 0

def increase_count():
    count += 1

increase_count()
```

Output: UnboundLocalError: cannot access local variable 'count' where it is not associated with a value

NOTE: Use global only when you really need to modify a global variable. Better style is usually to return a value:

Nishchal

```
PYTHON CODE
def increase_count(count):
    return count + 1

count = 0
count = increase_count(count)

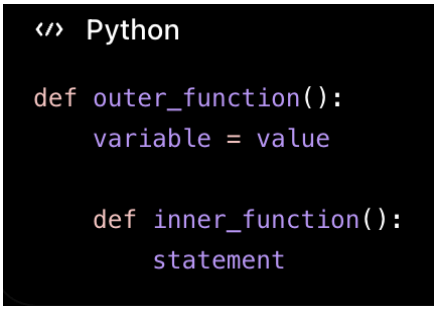
print(count)
```

Output: 1

12.6 Enclosing Scope

An **enclosing scope** exists when a function is defined inside another function.

Syntax



1. Enclosing scope belongs to the outer function.
2. Inner functions can access variables from the outer function.
3. This scope is between local and global scope.
4. Enclosing scope is important for nested functions and closures.

Example 1

```
PYTHON CODE
def outer():
    message = "Hello"

    def inner():
        print(message)

    inner()
outer()
```

Output: Hello

CodeWithNishchal

12.7 nonlocal Keyword

The nonlocal keyword is used to modify a variable from the nearest enclosing function scope.

Syntax

nonlocal variable_name

1. nonlocal is used inside nested functions. It allows the inner function to modify a variable from the outer function.
2. It does not work with global variables.
3. The variable must already exist in the enclosing function. nonlocal is commonly used in closures.

Example 1

PYTHON CODE

```
def outer():
    count = 0
    def inner():
        nonlocal count
        count += 1
        print(count)

    inner()
outer()
```

Output:1

Without nonlocal

PYTHON CODE

```
def outer():
    count = 0

    def inner():
        nonlocal count
        count += 1
        print(count)

    inner()

outer()
```

Output: UnboundLocalError: cannot access local variable 'count' where it is not associated with a value

leWithNishchal

global vs nonlocal

Keyword	Used For	Scope Affected
global	Modify global variable	Global scope
nonlocal	Modify enclosing function variable	Enclosing scope
Neither	Normal local variable	Local scope

12.8 Built-in Scope

Built-in scope contains names already provided by Python.
Examples: print, len, type, range, sum, max, min

Explanation

1. Built-in names are available automatically.
2. We do not need to define them.
3. Python searches built-in scope last in the LEGB rule.
4. Avoid using built-in names as variable names.

Example 1

```
numbers = [10, 20, 30]

print(len(numbers))
```

Output: 3

Bad Practice

```
list = [10, 20, 30]
```

This is bad because the list is already a built-in name. Do not use names like list, dict, str, int, sum, or max as variable names.

12.9 LEGB Rule

LEGB is the order Python follows while searching for a variable name.
L -> Local
E -> Enclosing
G -> Global
B -> Built-in

CodeWithNishchal

When Python sees a variable name, it searches in this order:

1. **Local scope**: inside the current function.
2. **Enclosing scope**: inside outer functions.
3. **Global scope**: main program/file.
4. **Built-in scope**: built-in Python names.

Example 1

```python
x = "global"
def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)

    inner()
outer()
```

Output: local ⇒ Python finds x in the local scope first, so it does not search further.

LEGB Search Table

Search Order	Scope	Example
1	Local	Variable inside current function
2	Enclosing	Variable inside outer function
3	Global	Variable outside functions
4	Built-in	print, len, type

12.10 Namespace Concept

A **namespace** is a system that stores names and their related objects.

Syntax Idea

name -> object

1. A namespace maps names to values or objects.
2. Python uses namespaces to avoid name conflicts.
3. Different scopes have different namespaces.
4. Local, global, and built-in scopes each have their own namespaces.
5. The same name can exist in different namespaces without conflict.

CodeWithNishchal

Example 1

```
PYTHON CODE
x = 100
def show():
    x = 50
    print(x)

show()
print(x)
```

Output: 50
100

1. The global namespace has x = 100. The local namespace inside show() has x = 50.
2. Both names are x, but they belong to different namespaces.

Namespace Table

Namespace	Contains
Local namespace	Names inside a function
Global namespace	Names created at file/program level
Built-in namespace	Python built-in names
Enclosing namespace	Names inside outer functions

12.11 locals() and globals()

Python provides two built-in functions to inspect namespaces.

Syntax

```
Python

locals()
globals()
```

1. locals() returns the current local namespace as a dictionary.
2. globals() returns the global namespace as a dictionary.
3. These are mainly used for debugging and learning.
4. Usually, we should not modify program logic using them.

codewithlishchal

Example 1

```
PYTHON CODE
course = "Python"

def show():
    name = "Aman"
    print(locals())

show()
```

Output: {'name': 'Aman'}

12.12 NameError & UnboundLocalError

These errors are common in scope-related topics.

NameError

NameError occurs when Python cannot find a name in LEGB search.

```
PYTHON CODE
print(age)
```

Output: NameError: name 'age' is not defined

UnboundLocalError

UnboundLocalError occurs when Python treats a variable as local because it is assigned inside a function, but it is used before assignment.

```
PYTHON CODE
x = 10
def show():
    print(x)
    x = 20

show()
```

output: UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

Error	Meaning
NameError	Name not found in any scope
UnboundLocalError	Local variable used before assignment

With Nishchal

12.13 Important Scope Summary Table

Concept	Meaning
Local scope	Variable inside current function
Global scope	Variable outside all functions
Enclosing scope	Variable inside outer function
Built-in scope	Python built-in names
global	Modifies global variable inside function
nonlocal	Modifies enclosing function variable
LEGB	Search order for names
Namespace	Mapping of names to objects
locals()	Shows local namespace
globals()	Shows global namespace
NameError	Name not found
UnboundLocalError	Local variable used before assignment

CodeWithNishchal